Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer

Raffel et al. (JMLR 2020), arXiv:1910.10683

Paper summary for the CS-781 capstone

The paper at a glance

Citation

C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, P. J. Liu. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. JMLR 21, 2020. Google.

One-sentence summary

Cast every NLP task as "text in, text out", then run a large controlled study of architectures, objectives, data, fine-tuning and scale. The result is **T5**.

Why it matters for our capstone

T5 directly compares encoder–decoder, prefix-LM and decoder-only models under equal budgets: our A2, A3 and A1. It also compares adapters with full fine-tuning (Phases 2–7).

Where it appears in our slides

Main deck: "The four architectures" and "A controlled experiment plan".

Roadmap

1. The problem and the framework
2. The systematic study
3. Assessment and outlook

Too many variables, too few controlled comparisons

By 2019, transfer learning (pre-train, then fine-tune) had many competing recipes: BERT, GPT, XLNet, MASS, UniLM...

Each paper changed several things at once: architecture, objective, data, model size, training length. It was unclear *which* choices actually mattered.

T5's approach:

1. One unified **text-to-text** format for all tasks.
2. A reasonable baseline.
3. Change **one factor at a time** and measure it on many tasks.
4. Combine the best choices and scale up.

Why this matters to us

Our Phase-1 study is the same kind of work on a smaller scale: four architectures, same data, same budget, one variable at a time. T5 shows how to design such a study, and gives a prediction for our A1-vs-A2-vs-A3 comparison.

The text-to-text framework

Every task becomes: input text (with a short **task prefix**) → output text. One model, one loss (maximum likelihood with teacher forcing), one decoding procedure.

Input	Target
translate English to German: That is good.	Das ist gut.
mnli premise: I hate pigeons. hypothesis: My feelings towards pigeons are filled with animosity.	entailment
summarize: *article*	*summary*
stsb sentence1: ... sentence2: ...	3.8

Easy legal examples

`judgment: <case text>` → `1`

`statutes: <case text>` → `IPC 302, IPC 120B`

`role: The accused was arrested on 3 May.` → `Fact`

`summarize: <judgment>` → `<summary>`

Even regression (a similarity score such as 3.8) is emitted as text.

A risk

The model could output an invalid label, e.g. "hamburger". It counts as wrong; the authors never saw it happen.

C4: the Colossal Clean Crawled Corpus

Common Crawl produces about 20 TB of text per month, mostly boilerplate, menus, code and gibberish. Filtering heuristics:

- keep lines ending in terminal punctuation; keep lines of 5 or more words; drop pages under 3 sentences;
- drop pages with bad words, “lorem ipsum”, or a curly bracket (code);
- drop lines mentioning Javascript or cookie and privacy policies;
- deduplicate repeated three-sentence spans;
- keep only pages detected as English with probability 0.99 or more.

The result is about **750 GB** of clean English text.

Lessons for our legal corpus

Our Phase-1 data is already legal text, but similar cleaning ideas apply:

- drop OCR fragments and page headers;
- deduplicate repeated boilerplate such as cause titles and standard clauses;
- drop tiny fragments (our `min_words` filter).

The baseline model and objective

- **Encoder–decoder**, each stack like BERT-base: 12 blocks, $d_{\text{model}} = 768$, $d_{ff} = 3072$, 12 heads. About **220M** parameters.
- Pre-LN with a scale-only LayerNorm (no bias). **Relative position bias**: a learned scalar per head and per distance bucket, 32 buckets, growing logarithmically up to 128 tokens.
- SentencePiece vocabulary of 32k; AdaFactor optimiser; inverse-square-root learning rate.
- Pre-training: 2^{19} steps $\times$ 2^{16} tokens $\approx$ **34B tokens**. No data is repeated.

Span-corruption objective

Drop 15% of tokens. Replace each dropped *span* with a unique sentinel. The target lists only the dropped spans.

Input: "Thank you <X> me to your party <Y> week."
Target: "<X> for inviting <Y> last <Z>"

Short targets make pre-training cheap.

Does pre-training help, and how noisy are results?

	GLUE	CNNDM	SQuAD	SGLUE
Baseline (10 runs)	83.28	19.24	80.88	71.36
Std. deviation	0.235	0.065	0.343	0.416
No pre-training	66.22	17.60	50.31	53.04

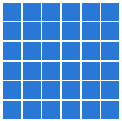
Pre-training gives large gains everywhere except where data is plentiful (e.g. EnFr translation, 39.82 vs 39.77).

A good habit to copy

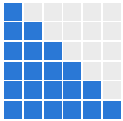
The authors trained the baseline **10 times** to measure run-to-run noise. A later change that moves GLUE by only 0.2 is within one standard deviation and may be noise.

For our capstone: repeat the baseline with 2–3 seeds if possible, before claiming a small gain.

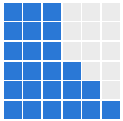
Architectures compared: it is all about the mask



fully visible
(encoder)



causal
(decoder / LM)



causal with prefix
(prefix LM)

Structures compared:

- **Encoder–decoder**: fully visible encoder, causal decoder, cross-attention. Our A2.
- **Encoder–decoder, shared**: the same weights in both stacks.
- **Language model**: one causal stack over input + target. Our A1 and A4.
- **Prefix LM**: one stack; fully visible over the input, causal over the target. Our A3.

Fair comparison: an encoder–decoder with $L + L$ layers has $2P$ parameters but about the *same cost* M as an L-layer LM on input + target, because each token passes through only one stack.

Architecture results

Architecture	Objective	Params	Cost	GLUE	CNNDM	SGLUE
Encoder–decoder	Denoising	$2P$	M	**83.28**	**19.24**	**71.36**
Enc–dec, shared	Denoising	P	M	82.81	18.78	70.73
Enc–dec, 6 layers	Denoising	P	$M/2$	80.88	18.97	68.42
Language model	Denoising	P	M	74.70	17.93	55.02
Prefix LM	Denoising	P	M	81.82	18.61	68.11
Encoder–decoder	LM	$2P$	M	79.56	18.59	64.29
Language model	LM	P	M	73.78	17.54	56.51
Prefix LM	LM	P	M	79.68	17.84	64.86

Takeaways

Encoder–decoder wins. Sharing its weights costs little (83.28 to 82.81) while halving parameters. **Prefix LM beats the plain LM** by a wide margin, because the input is fully visible. Denoising beats plain LM objectives.

Downstream fine-tuning scores, not next-token perplexity; our Phase 1 measures continuation quality, so the ranking may differ.

Unsupervised objectives

Variants of BERT-style denoising in the encoder–decoder:

Objective	GLUE	CNNDM	SGLUE
BERT-style	82.96	19.17	69.85
MASS-style	82.32	19.16	69.28
Replace corrupted spans	83.28	19.24	**71.36**
Drop corrupted tokens	**84.44**	**19.31**	68.67

Also tested: corruption rates of 10–50% and average span lengths of 2–10. Differences were small; 50% corruption was worst.

Takeaway

Most denoising objectives perform **similarly**. So choose the one with the *shortest targets*, which trains fastest.

Span corruption with average span length 3 slightly but significantly beat independent token corruption on most non-translation tasks.

Pre-training data: domain and size

Data set	Size	GLUE	SQuAD	SGLUE
C4	745 GB	83.28	80.88	71.36
C4, unfiltered	6.1 TB	81.46	78.78	68.04
RealNews-like	35 GB	83.83	80.39	72.38
WebText-like	17 GB	84.03	81.42	71.40
Wikipedia	16 GB	81.85	81.29	68.01
Wikipedia + books	20 GB	83.65	**82.08**	**73.24**

Repeating a small data set over the same 34B tokens:

Unique tokens	Repeats	GLUE	SGLUE
full C4	0	83.28	71.36
2^{27}	256	82.62	69.97
2^{23}	4,096	76.34	59.29

Takeaways

- **Cleaning matters**: unfiltered C4 is worse, although 8× bigger.
- **In-domain data helps in-domain tasks**. Book text boosted a fiction-based reading task (MultiRC exact match 25.8 to 50.9).
- Heavy repetition hurts; the model memorises.

For us: legal pre-training should help legal tasks. Our budget of about 200M tokens from 4.5B available means **no repetition** needed.

Fine-tuning strategies and multi-task learning

Updating only part of the model:

Method	GLUE	CNNDM	EnDe
All parameters	**83.28**	**19.24**	**26.98**
Adapters, $d = 32$	80.52	15.08	13.84
Adapters, $d = 512$	81.54	17.78	23.45
Gradual unfreezing	82.50	18.95	26.71

Multi-task: training on a mixture of tasks with various mixing rates never quite matched pre-train-then-fine-tune. Multi-task pre-training followed by fine-tuning was comparable.

For Phases 2–7

- Full fine-tuning was best, but most expensive.
- Small adapters struggle on **generation-heavy** tasks (summarisation, translation); enough capacity is needed.
- Adapters with $d = 32$ are fine for classification (GLUE 80.5) but poor for translation (13.8 BLEU).

Expect PEFT gaps to be larger in Phase 6 (summarisation) than in Phases 3–5.

Scaling: how to spend 4× more compute

Strategy	GLUE	SGLUE	EnDe
Baseline	83.28	71.36	26.98
1× size, 4× steps	85.33	74.72	27.08
1× size, 4× batch	84.60	74.64	27.07
2× size, 2× steps	**86.18**	77.18	**27.52**
4× size, 1× steps	85.91	**78.04**	27.47
4× ensembled	84.77	71.74	28.05

Takeaways

- Every option helps.
- A **bigger model trained for fewer steps** usually beats a smaller model trained longer.
- Ensembles give an independent boost.

For us: within 50–300M parameters and a fixed Colab budget, test a larger model for fewer steps against a smaller one for more steps.

Putting it all together: the T5 models

Final recipe: encoder–decoder, span corruption (average span 3), C4, multi-task pre-training then per-task fine-tuning, and much more compute: over **1 trillion tokens** in total. Sizes from Small (60M) to 11B.

Model	CNN/DM ROUGE-2	CNN/DM ROUGE-L
Previous best	20.30	40.63
T5-Small	19.56	38.35
T5-Base	20.34	39.40
T5-Large	20.68	39.75
T5-11B	**21.55**	**40.69**

Headline results

- SuperGLUE: **88.9**, up from 84.6, close to the human score of 89.8.
- State of the art on many GLUE, SuperGLUE, SQuAD and CNN/DM measures.
- **Not** state of the art on WMT translation, probably because pre-training was English-only and without back-translation.

What T5 predicts for our Phase-1 study

T5 finding	Hypothesis for our capstone
Encoder–decoder best for fine-tuning	A2 may transfer best to Phases 2–7, even if not best at continuation
Prefix LM $\gg$ plain LM	A3 should beat A1 when the prompt is long and informative
Shared enc–dec nearly as good	Parameter sharing (A3) is a cheap way to get encoder benefits
Denoising $>$ LM objective downstream	Our models all use LM-style objectives; span corruption is a possible extra experiment
In-domain data helps	Legal-only pre-training should help legal tasks; test mixing ratios
Bigger model, fewer steps	Compare 80M and 150M models at equal compute
Run-to-run noise	Repeat key runs with different seeds

Strengths and limitations

Strengths

- Very large, carefully controlled study; one variable at a time.
- Measures run-to-run variance.
- Simple unified format for every task.
- Released C4, code and all model checkpoints.
- Clear, practical takeaways.

Limitations

- Enormous compute; hard to reproduce outside large labs.
- Factors are studied one at a time, so interactions can be missed.
- English-only pre-training.
- Judged on fine-tuning benchmarks; open-ended generation quality was not the focus.
- Maximum length 512: long documents not studied.

Future research directions (1): from the paper's outlook and after

Direction	Details
Stronger small models *(paper)*	Distillation, parameter sharing, conditional computation for low-resource settings
More efficient knowledge extraction *(paper)*	Denoising may be inefficient; ELECTRA (Clark et al., 2020) trains a discriminator instead
Formalise task similarity *(paper)*	A principled way to choose pre-training data for a target task
Language-agnostic models *(paper)*	mT5 (Xue et al., 2021) trains T5 on 101 languages
Instruction tuning	Flan-T5 (Chung et al., 2022) fine-tunes on over 1,800 tasks phrased as instructions
Better objective mixtures	UL2 (Tay et al., 2023) mixes prefix-LM and several span-corruption denoisers
Tokenizer-free T5	ByT5 (Xue et al., 2022) works directly on bytes

Future research directions (2): open questions

Open in general

- Which architecture is best when the goal is open-ended generation *and* fine-tuning?
- How factors interact: is the best objective different at large scale?
- Long-input text-to-text models.
- Cheaper ways to reach the same downstream quality.

Open for legal NLP

- One text-to-text legal model for all seven phases, using task prefixes.
- Text outputs for span tasks (NER) need exact character offsets, a known weak spot of text-to-text.
- Legal-specific cleaning heuristics, the equivalent of C4 for judgments.

Summary

- **Text-to-text**: every task is input text $\rightarrow$ output text, trained with one loss.
- **C4**: 750 GB of heuristically cleaned web text.
- Controlled findings: encoder–decoder best; prefix LM much better than plain LM; denoising beats LM objectives downstream; objectives are similar, so use short targets; clean and in-domain data help; heavy repetition hurts; full fine-tuning beats adapters; bigger models beat longer training.
- Scaled to 11B parameters: state of the art on many benchmarks, SuperGLUE 88.9 vs human 89.8.
- For us: a template for a controlled architecture study, and concrete predictions for A1–A3.

Reference

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research*, 21(140):1–67. arXiv:1910.10683.

Later work cited: Clark et al. (2020) ELECTRA; Xue et al. (2021) mT5; Chung et al. (2022) Scaling Instruction-Finetuned Language Models (Flan-T5); Xue et al. (2022) ByT5; Tay et al. (2023) UL2.